# 4.1 Linear regression

Vladimir Buskin

English Language and Linguistics, Catholic University of Eichstätt-Ingolstadt

ORCID: 0009-0005-5824-1012

Last updated: 14 October 2026, 21:40

This handout introduces linear regression for modelling continuous response variables, covering least squares estimation, the interpretation of model output, model visualisation, assumption checks and the evaluation of model performance via test data and cross-validation, using reaction time data from the English Lexicon Project.

## Table of contents

Recommended reading . . . 2
Preparation . . . 2
Introduction . . . 3
Descriptive overview . . . 3
- Log-transformed . . . 3
- Untransformed . . . 4
- A simple statistical model . . . 6

Linear regression . . . 7
- Data splitting . . . 7
- Model with a single predictor $X$ . . . 7
- Ordinary least squares . . . 8
- Application in R . . . 10
- Multiple linear regression . . . 13
- Application in R . . . 15

Visualising regression models . . . 16
Model assumptions and diagnostics . . . 19
Model performance . . . 22
- Test error . . . 22
- Cross-validation . . . 24

Exercises . . . 26
- Tier 1 . . . 26
- Tier 2 . . . 26

Tier 3 . . . . . 27
References . . . . . 29

## Recommended reading

For linguists:

Levshina (2015: Chapter 7)

Winter (2020: Chapter 4)

General:

Heumann et al. (2022: Chapter 11)

James et al. (2021: Chapter 3)

Hastie et al. (2017: Chapter 3)

## Preparation

> **Script**
>
> You can find the full R script associated with this unit here.

```
# Load libraries
library(readxl)
library(tidyverse)
library(rsample) # for obtaining training and test data
library(broom) # converting models to data frames
library(ggeffects) # generating predictions (ggeffect() additionally
 ↪  requires the "effects" package to be installed)
library(car) # model diagnostics (VIF etc.)
library(caret) # model training

# Load data
ELP <- read_xlsx("ELP.xlsx")

# Inspect data structure
str(ELP)
```

## Introduction

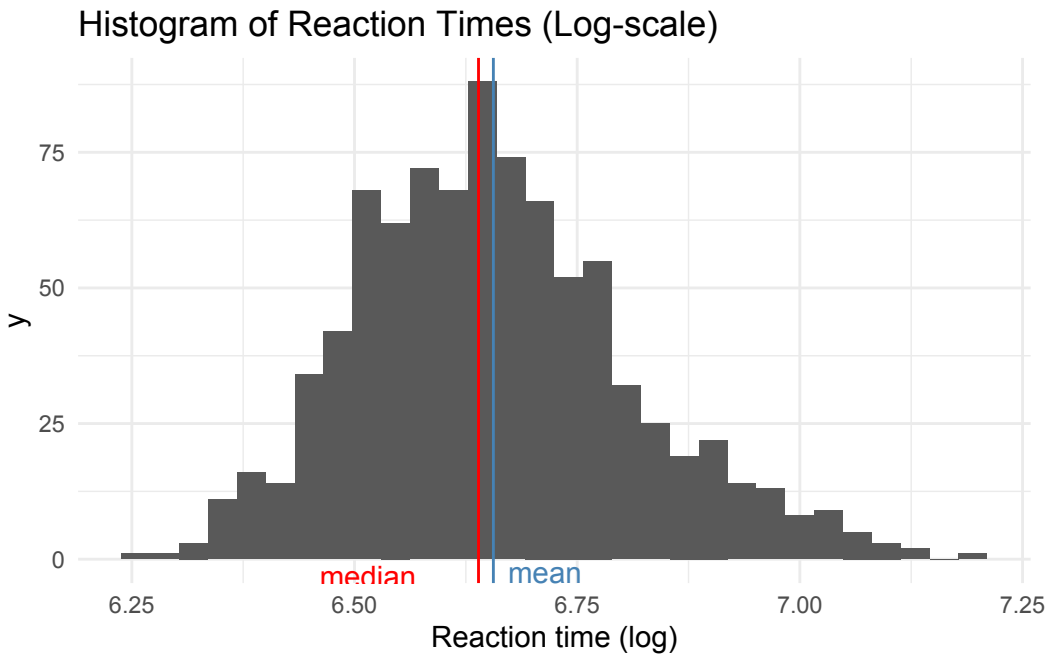
The `ELP` (English Lexicon Project) dataset (Balota et al. 2007) comprises reaction times (`RT`) on 880 English lemmas (`Word`). For each item, the data includes information on the part of speech (`POS`), normalised frequency (`Freq`), and the length in letters (`Length`).

## Descriptive overview

### Log-transformed

```
# Log-transformed
ggplot(ELP, aes(x = log(RT))) +
  geom_histogram() +
  geom_vline(xintercept = mean(log(ELP$RT)), color = "steelblue") +
  geom_vline(xintercept = median(log(ELP$RT)), color = "red") +
  theme_minimal() +
  labs(
    title = "Histogram of Reaction Times (Log-scale)",
    x = "Reaction time (log)"
  ) +
   annotate("text", x = mean(log(ELP$RT)), y = 0,
            label = "mean",
            vjust = 1.5, hjust = -0.2, color = "steelblue", parse = TRUE) +
  annotate("text", x = median(log(ELP$RT)) - 0.2, y = .7,
            label = "median",
            vjust = 1.5, hjust = -0.2, color = "red", parse = TRUE)
```

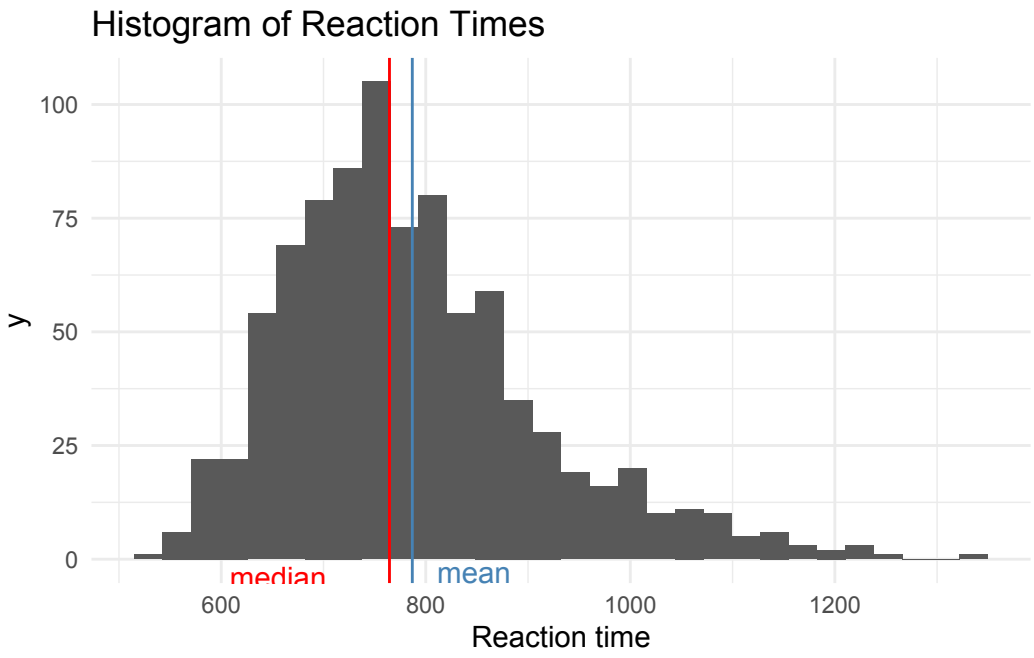
```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

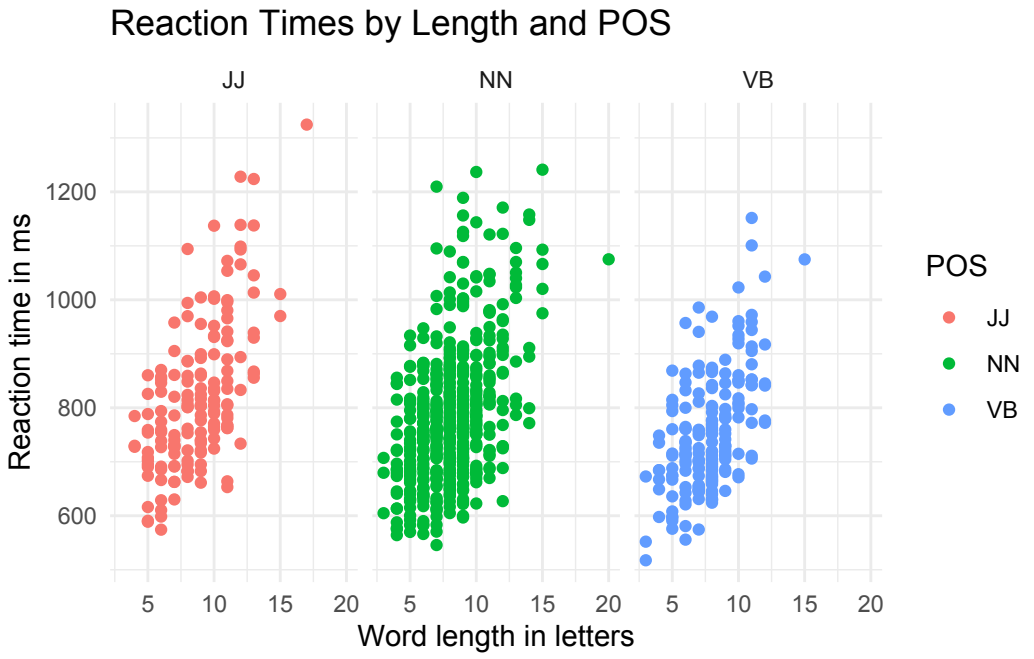


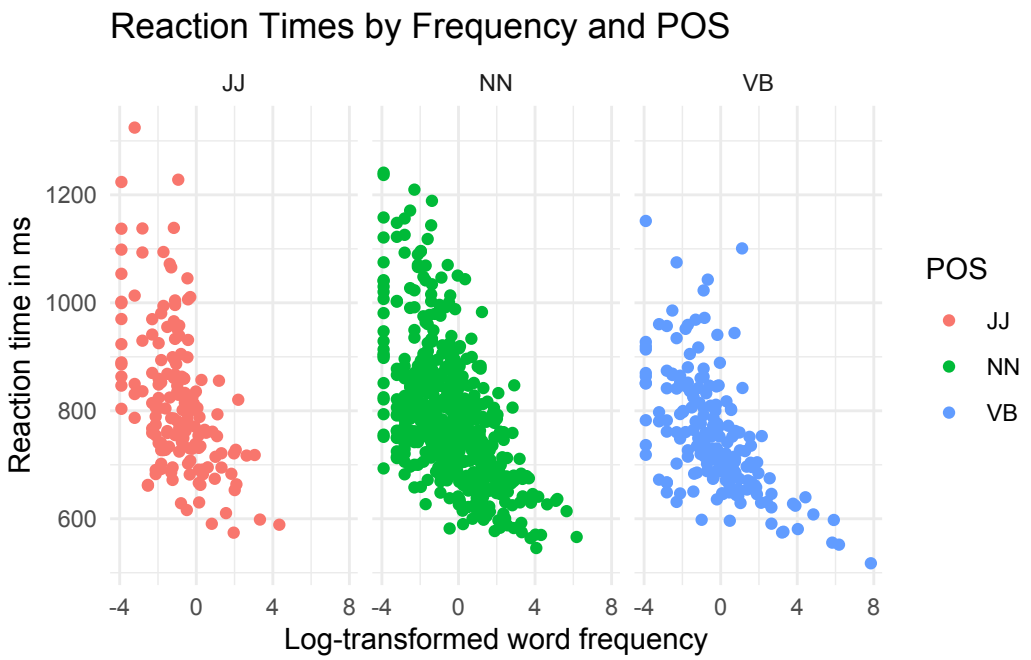

**Untransformed**

```
# Skewed
ggplot(ELP, aes(x = RT)) +
  geom_histogram() +
  geom_vline(xintercept = mean(ELP$RT), color = "steelblue") +
  geom_vline(xintercept = median(ELP$RT), color = "red") +
  theme_minimal() +
  labs(
    title = "Histogram of Reaction Times",
    x = "Reaction time"
  ) +
annotate("text", x = mean(ELP$RT) + 10, y = 0,
           label = "mean",
           vjust = 1.5, hjust = -0.2, color = "steelblue", parse = TRUE) +
  annotate("text", x = median(ELP$RT) -175, y = .7,
           label = "median",
           vjust = 1.5, hjust = -0.2, color = "red", parse = TRUE)
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```


Histogram of Reaction Times
100
75
50
25
0
y
median
mean
600
800
1000
1200
Reaction time


Reaction Times by Length and POS
JJ
NN
VB
1200
1000
800
600
Reaction time in ms
5
10
15
20
Word length in letters
POS
JJ
NN
VB




Some open questions

- Can word frequencies, word length and POS help us **explain variation** in reaction times?
- If they can, then how could we characterise the **effects** of these independent variables? In other words, do they increase or decrease reaction times?
- What reaction times could we expect for **new observations**?

### A simple statistical model

The dependent variable `RT` is the *response* or *target* that we wish to explain. We generically refer to the response as $Y$. Independent variables, such as `Length`, are called *features* or *predictors*, and are typically denoted by $X$.

We can now posit a fairly general statistical model of the form:

$$Y = f(X) + \epsilon. \tag{1}$$

The term $f(X)$ describes the contribution of an independent variable $X$ to the explanation of $Y$. Since hardly any model can explain the data perfectly, we expect there to be some degree of error $\epsilon$.

## Linear regression

Linear regression is a simple approach to supervised machine learning where the **continuous** response variable is known.[1] The predictors may be continuous or categorical.

One of the core assumptions of linear regression models is that the dependence of $Y$ on $X$ is **linear**, i.e., their relationship is a straight line. Although it may seem overly simplistic, linear regression is extremely useful both conceptually and practically.

### Data splitting

First, we split the data into **training data** used for model fitting and **test data**, which will provide an indication of how well the model can generalise to new observations.[2]

```
# Set random seed to ensure we will always get the "same" random numbers
set.seed(456)

# Assign 3/4 of the data to the training set
data_split <- initial_split(ELP, prop = 3/4)

# Create data frames for the two sets:
train_data <- training(data_split)

test_data  <- testing(data_split)
```

### Model with a single predictor $X$

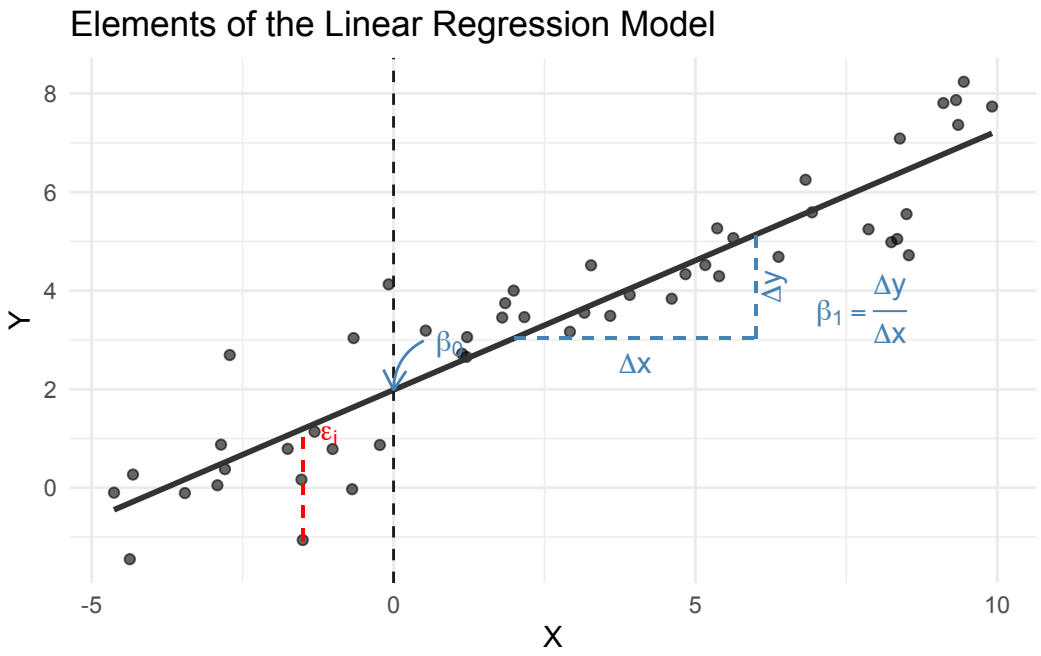
The simple linear model has the general form

$$Y = \beta_0 + \beta_1 X + \epsilon. \tag{2}$$

- The model parameters (or coefficients) $\beta_0$ and $\beta_1$ specify the functional relationship $f$ between $Y$ and $X$.
- The first parameter $\beta_0$ determines the **intercept** of the regression line, and $\beta_1$ indicates the **slope**.
- Once again, $\epsilon$ captures the model error. For each observation $i$, the error $\epsilon_i$ corresponds to the vertical distance between the data point and the regression line.

[1] If the response variable is unknown or irrelevant, we speak of **unsupervised machine learning**. Unsupervised models are mostly concerned with finding patterns in high-dimensional datasets with dozens or even hundreds of variables.

[2] For details on the rationale behind train/test splits, see James et al. (2021: 29–33).




Suppose we are interested in modelling the relationship between `RT` and `Length`. Then the updated model equation would have the form

$$\text{Reaction time} = \beta_0 + \beta_1 \text{Length} + \text{Model Error}.$$

But how do we find the exact values of the intercept and the slope? In short: We can't! We are dealing with population parameters and can, therefore, only estimate the true relationship between `RT` and `Length` from our sample.

To reflect the tentative nature of the model coefficients, we use the **hat** symbol $\widehat{\ }$ (e.g., $\hat{\beta}_0$) to indicate **estimations** rather than true values. The estimation procedure requires **training data**, based on which the algorithm "learns" the relationship between $Y$ and $X$ (hence the term "Machine Learning").

### Ordinary least squares

The most common way of estimating parameters for linear models is the **Least Squares** approach. In essence, the parameters are chosen such that the residual sum of squares (RSS), i.e., the sum of the squared differences between observed and predicted values, is as low as possible:

$$\min_{\beta_0,\beta_1} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2. \tag{3}$$

In other words, we are trying to minimise the (squared vertical) distances between the data points and the regression line.

The slope can be computed using the equivalence in Equation 4:

$$\hat{\beta}_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}. \tag{4}$$

```
# Define X and Y
x <- train_data$Length
y <- train_data$RT

# Compute means
x_bar <- mean(x)
y_bar <- mean(y)

# Compute beta_1 (slope)
beta_1 <- sum((x - x_bar) * (y - y_bar)) / sum((x - x_bar)^2)

print(beta_1)
```

```
[1] 27.9232
```

We can then obtain the intercept via Equation 5.

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \tag{5}$$

```
# Compute beta_0 (intercept)
beta_0 <- y_bar - beta_1 * x_bar

print(beta_0)
```

```
[1] 555.0913
```

Once we've fitted the model, we can then predict reaction times if we know the length of a lexical stimulus:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x, \tag{6}$$

where $\hat{y}$ indicates a prediction of $Y$ on the basis of the predictor values $X = x$. For example, what reaction time would we expect to see for a long word with 14 letters?

```
# Make predictions
y_hat <- beta_0 + beta_1 * 14

print(y_hat)
```

```
[1] 946.0161
```

### Application in R

While it is certainly possible to run a linear regression by hand, R's `lm()` function provides a faster method that additionally computes an abundance of helpful statistics.

```
# Fit linear model
rt.lm1 <- lm(RT ~ Length, data = train_data)

# View model data
summary_rt1 <- summary(rt.lm1)

print(summary_rt1)
```

```
Call:
lm(formula = RT ~ Length, data = train_data)

Residuals:
    Min      1Q  Median      3Q     Max
-263.11  -79.54  -16.04   63.69  459.12

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  555.091     15.247   36.41   <2e-16 ***
Length        27.923      1.795   15.56   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 106.6 on 658 degrees of freedom
Multiple R-squared:  0.2689,    Adjusted R-squared:  0.2678
F-statistic:   242 on 1 and 658 DF,  p-value: < 2.2e-16
```

The model statistics comprise the following elements:

> **Call**
>
> i.e., the model formula.

> **Residuals**
>
> These indicate the difference between the observed values in the data set and the values predicted by the model (= the fitted values). They are our estimates of the error term $\epsilon$. The smaller the residuals (in absolute terms), the better the model describes the

data.

```
# Show fitted values (= predictions) for the first six observations
head(rt.lm1$fitted.values)
```

```
       1        2        3        4        5        6
778.4769 834.3233 890.1697 946.0161 750.5537 722.6305
```

```
# Show deviation of the fitted values from the observed values
head(rt.lm1$residuals)
```

```
         1          2          3          4          5          6
-122.69690   21.98671  -94.37968 -174.40607  207.29630 -140.63051
```

```
# Show true values in the training data
head(train_data$RT)
```

```
[1] 655.78 856.31 795.79 771.61 957.85 582.00
```

**Coefficients**

The regression coefficients correspond to $\hat{\beta}_0$ ("Intercept") and $\hat{\beta}_1$ ("Length"), respectively. The model shows that for each additional letter, the predicted reaction time increases by approx. 28 ms.

```
# Convert coefficients to a tibble (= tidyverse-style data frame)
tidy_model <- tidy(rt.lm1)

tidy_model
```

```
# A tibble: 2 x 5
  term        estimate std.error statistic   p.value
  <chr>          <dbl>     <dbl>     <dbl>     <dbl>
1 (Intercept)    555.      15.2       36.4 8.40e-160
2 Length          27.9      1.79      15.6 1.05e- 46
```

***p*-values and *t*-statistic**

The model tests the null hypothesis $H_0$ that there is no relationship between `RT` and `Length` (i.e., $H_0 : \beta_1 = 0$). Since $p < 0.001$, we can reject $H_0$.
The $p$-value itself crucially depends on the $t$-statistic, which measures "the number of

standard deviations that $\hat{\beta}_1$ is away from 0" (James et al. 2021: 67). The standard error (SE) reflects how much an estimated coefficient would vary on average across repeated samples, i.e., how far it typically lies from the true value. Standard errors can be used to compute an approximate 95% confidence interval:

$$[\hat{\beta}_1 - 2 \cdot SE(\hat{\beta}_1), \hat{\beta}_1 + 2 \cdot SE(\hat{\beta}_1)]. \tag{7}$$

If we were to sample from the population repeatedly and compute such an interval each time, 95% of these intervals would contain the true value of $\beta_1$. For example, the estimated slope parameter for `Length`, which is 27.9, has the 95% CI [24.4, 31.4].

```
# Compute confidence intervals for intercept and Length
tidy_model_ci <- tidy(rt.lm1, conf.int = TRUE)

tidy_model_ci
```

```
# A tibble: 2 x 7
  term        estimate std.error statistic   p.value conf.low conf.high
  <chr>          <dbl>     <dbl>     <dbl>     <dbl>    <dbl>     <dbl>
1 (Intercept)    555.      15.2       36.4 8.40e-160    525.      585.
2 Length          27.9      1.79      15.6 1.05e- 46     24.4      31.4
```

### Residual standard error (RSE)

This is an estimate of the standard deviation of the error term $\epsilon$, i.e., roughly the typical deviation of the observed values from the predictions (for a model with a single predictor).

$$RSE = \sqrt{\frac{1}{n-2}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{8}$$

```
summary_rt1$sigma
```

```
[1] 106.5928
```

### $R^2$

The $R^2$ score is important for assessing model fit because it "measures the proportion of variability in $Y$ that can be explained using $X$" (James et al. 2021: 70), varying between 0 and 1.

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{9}$$

Here, RSS denotes the residual sum of squares and TSS the total sum of squares.

```
summary_rt1$r.squared
```

```
[1] 0.2689159
```

The adjusted $R^2_{\text{adj}}$ measure imposes an additional penalty on the model for including many predictors. It never exceeds the unadjusted $R^2$.

```
summary_rt1$adj.r.squared
```

```
[1] 0.2678048
```

*F*-statistic

It tests whether the model as a whole explains a significant amount of variation in the response, i.e., the null hypothesis that all slope coefficients are zero ($H_0 : \beta_1 = \beta_2 = \cdots = \beta_p = 0$). Under $H_0$, $F$ should be close to 1; much larger values indicate that at least one predictor is associated with the response. With a single predictor, this is equivalent to $H_0 : \beta_1 = 0$. The $F$ statistic is computed as shown below (cf. Agresti and Kateri 2022: 232), where $p$ denotes the number of predictors, and follows an $F$-distribution with $p$ and $n - (p + 1)$ degrees of freedom.

$$F = \frac{(TSS - RSS)/p}{RSS/[n - (p + 1)]} \tag{10}$$

```
summary_rt1$fstatistic # F-value with numerator and denominator degrees
↪  of freedom; these parameters are needed for computing the p-value
```

```
   value    numdf    dendf
242.0332   1.0000 658.0000
```

### Multiple linear regression

In multiple linear regression, more than one predictor variable is taken into account. For instance, modelling `RT` as a function of `Length`, `Freq` and `POS` requires a more complex model of the form

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_p X_p + \epsilon. \tag{11}$$

In more concrete terms, we need to find the slope coefficients for all predictors in

$$Y = \beta_0 + \beta_1 \text{Length} + \beta_2 \text{Freq} + \beta_3 \text{POS} + \epsilon,$$

which **jointly minimise** the squared distances between the data points and the fitted values.[3] Doing so requires matrix algebra.

Least squares in matrix form

Let $\mathbf{y}$ denote the response vector

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{bmatrix},$$

which contains the outcomes in the training data.

```
# Response vector y
y <- train_data$RT
```

The predictors are stored in the $n \times (p+1)$ feature (or design) matrix $\mathbf{X}$, whose first column of 1s corresponds to the intercept:

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}$$

```
# Design matrix X
# Note that categorical variables are converted to binary indicator
 ↪  variables (1 = yes vs. 0 = no; also known as "dummy variables").
X <- model.matrix(RT ~ Length + Freq + POS, data = train_data)

head(X)
```

```
  (Intercept) Length Freq POSNN POSVB
1           1      8 4.31     1     0
2           1     10 1.04     1     0
3           1     12 0.18     1     0
4           1     14 0.06     1     0
5           1      7 0.41     0     0
6           1      6 0.63     1     0
```

We are looking for the parameter vector

[3] Strictly speaking, a categorical predictor such as `POS` with $k$ levels is represented by $k-1$ binary indicator variables, each with its own coefficient (see the design matrix below).

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \dots \\ \beta_p \end{bmatrix}$$

that minimises the residual sum of squares. The matrix form of the linear regression equation can thus be described as

$$\mathbf{y} = \mathbf{X}\beta + \epsilon. \tag{12}$$

After some rearrangement and differentiation, the closed-form solution for the optimal model parameters can be obtained as follows:

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \tag{13}$$

```
# Apply the matrix formula
# %*% indicates matrix multiplication
# t() transposes (switches rows and columns of) a matrix (i.e. X^T)
# solve() computes the matrix inverse (here: of X^T X)
beta_hat <- solve(t(X) %*% X) %*% t(X) %*% y

# Print coefficient vector
print(beta_hat)
```

```
                    [,1]
(Intercept) 583.41426086
Length       26.84458313
Freq         -0.08548284
POSNN       -17.78463311
POSVB       -38.05952430
```

### Application in R

In R, a multiple regression model is fitted as in the code example below:

```
# Fit multiple regression model
rt.lm2 <- lm(RT ~ Length + Freq + POS, data = train_data)

# View model statistics
summary(rt.lm2)
```

```
Call:
lm(formula = RT ~ Length + Freq + POS, data = train_data)
```

```
Residuals:
    Min      1Q  Median      3Q     Max
-260.46  -77.05  -15.67   62.41  456.14

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 583.41426   18.16955  32.109  < 2e-16 ***
Length       26.84458    1.80506  14.872  < 2e-16 ***
Freq         -0.08548    0.04017  -2.128  0.03370 *
POSNN       -17.78463   10.84663  -1.640  0.10156
POSVB       -38.05952   13.09292  -2.907  0.00377 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 105.7 on 655 degrees of freedom
Multiple R-squared:  0.2844,    Adjusted R-squared:   0.28
F-statistic: 65.08 on 4 and 655 DF,  p-value: < 2.2e-16
```

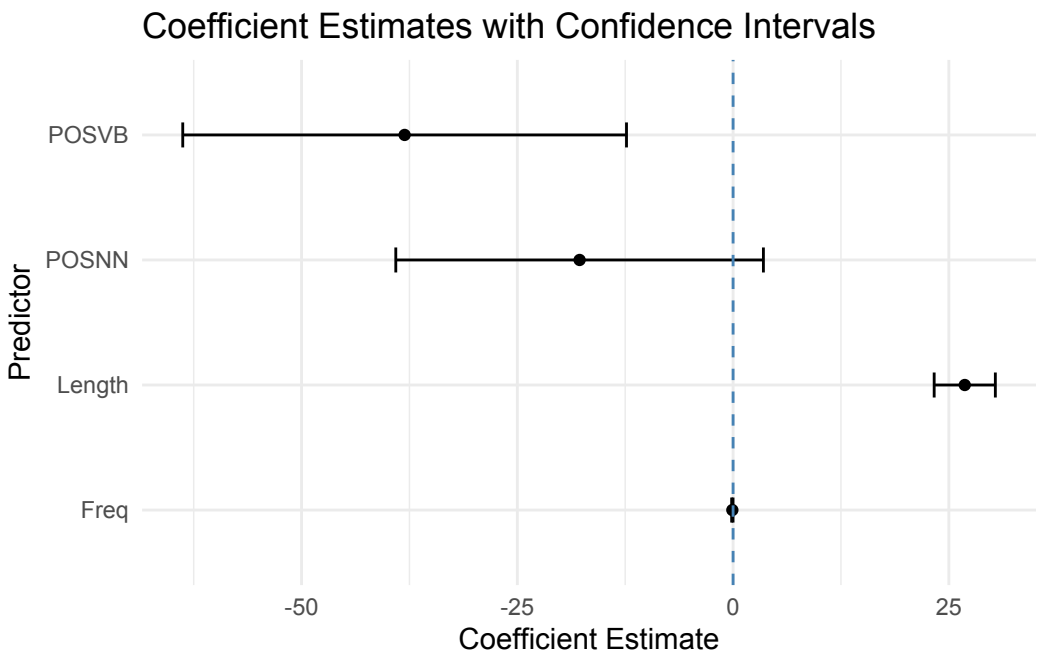
### Visualising regression models

- Plot coefficient estimates:

```r
# Tidy the model output
tidy_model <- tidy(rt.lm2, conf.int = TRUE)

# Remove intercept
tidy_model <- tidy_model %>% filter(term != "(Intercept)")

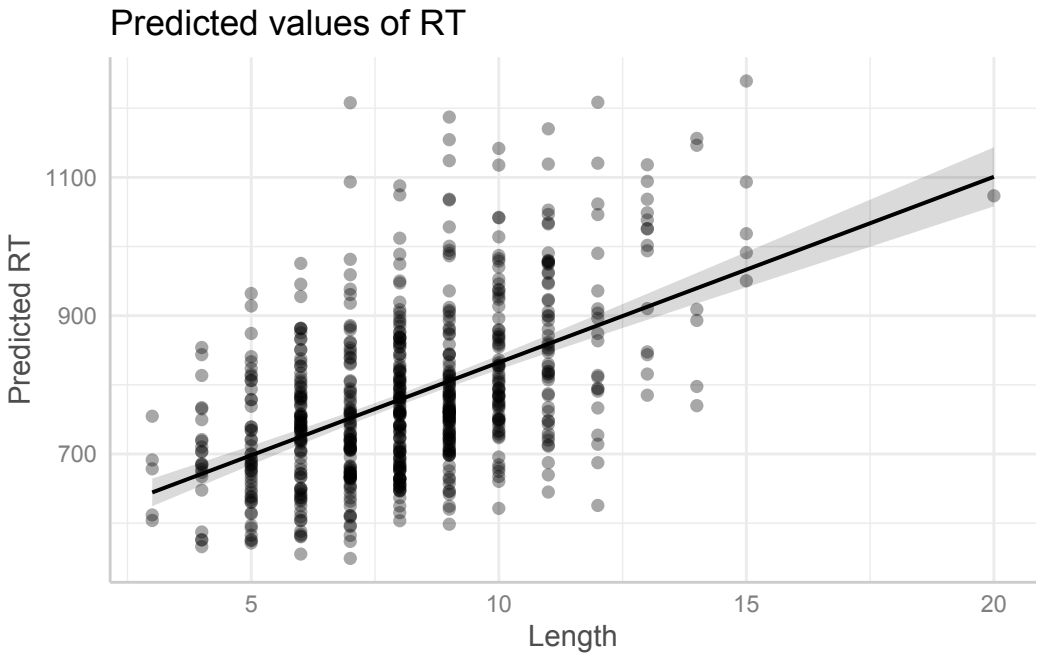
# Create the coefficient plot
ggplot(tidy_model, aes(x = estimate, y = term)) +
  geom_point() +
  geom_errorbarh(aes(xmin = conf.low, xmax = conf.high), height = 0.2) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "steelblue") +
  theme_minimal() +
  labs(
    x = "Coefficient Estimate",
    y = "Predictor",
    title = "Coefficient Estimates with Confidence Intervals"
  )
```




- Plot predictions:

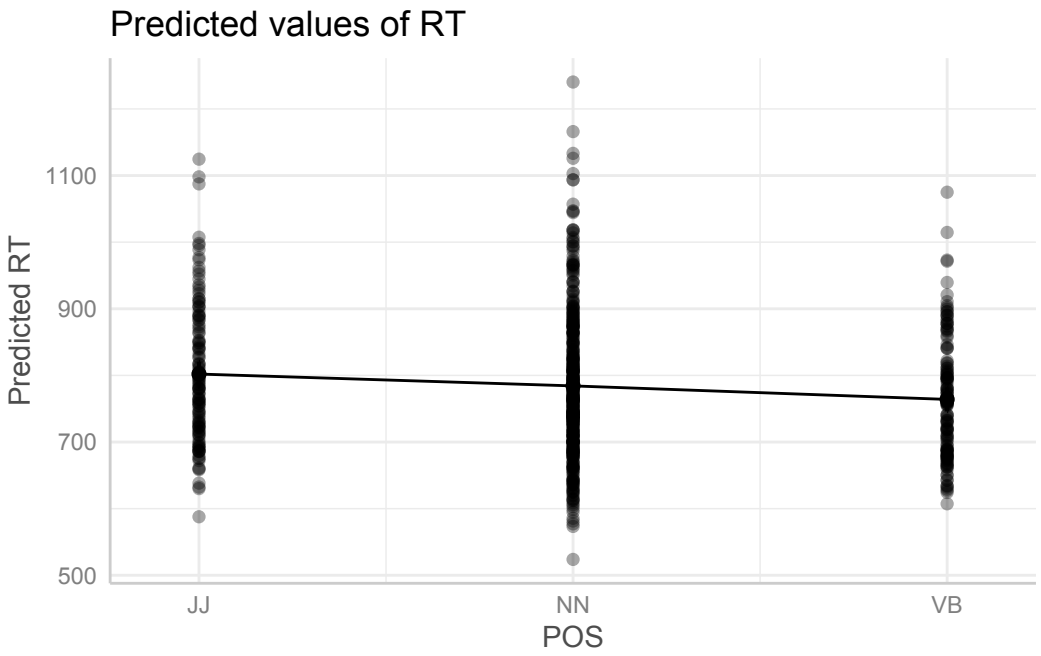
```
plot(ggeffect(rt.lm2, "Length"), show_residuals = TRUE) +
  labs(y = "Predicted RT")
```

```
Data points may overlap. Use the `jitter` argument to add some amount of
  random variation to the location of data points and avoid overplotting.
```



```
plot(ggeffect(rt.lm2, "POS"), show_residuals = TRUE) +
  geom_line() +
  labs(y = "Predicted RT")
```

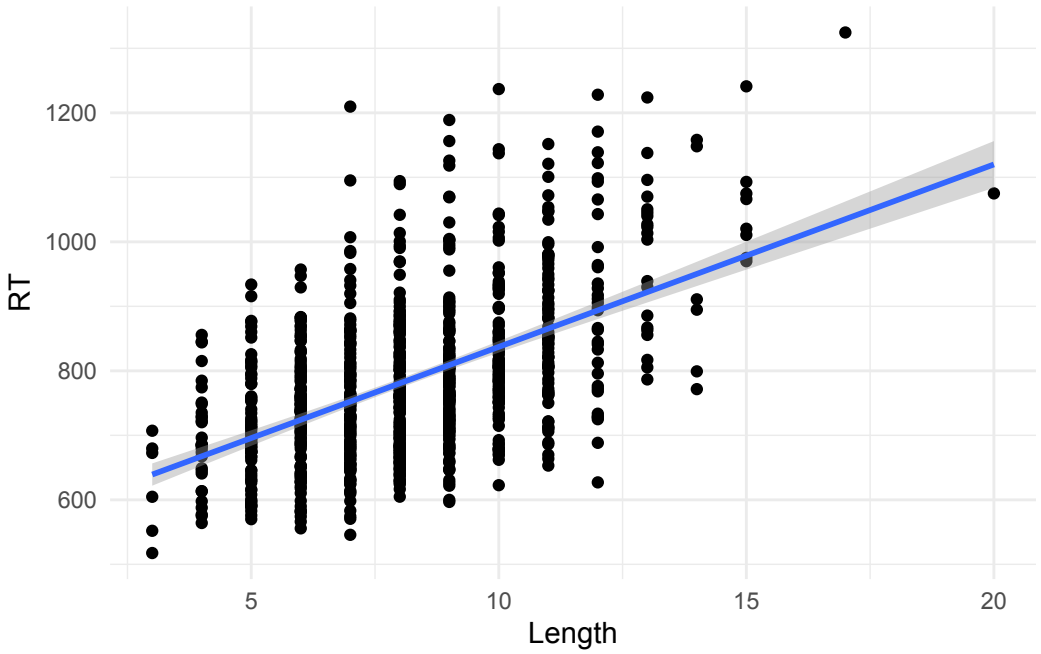
```
Data points may overlap. Use the `jitter` argument to add some amount of
  random variation to the location of data points and avoid overplotting.
```




- Using `ggplot2`'s `geom_smooth()` function, which adds a simple regression line for a single predictor (note that this is not the multiple regression model `rt.lm2`):

```
ggplot(ELP, aes(x = Length, y = RT)) +
  geom_point() +
  geom_smooth(method = "lm") +
  theme_minimal()
```

```
`geom_smooth()` using formula = 'y ~ x'
```



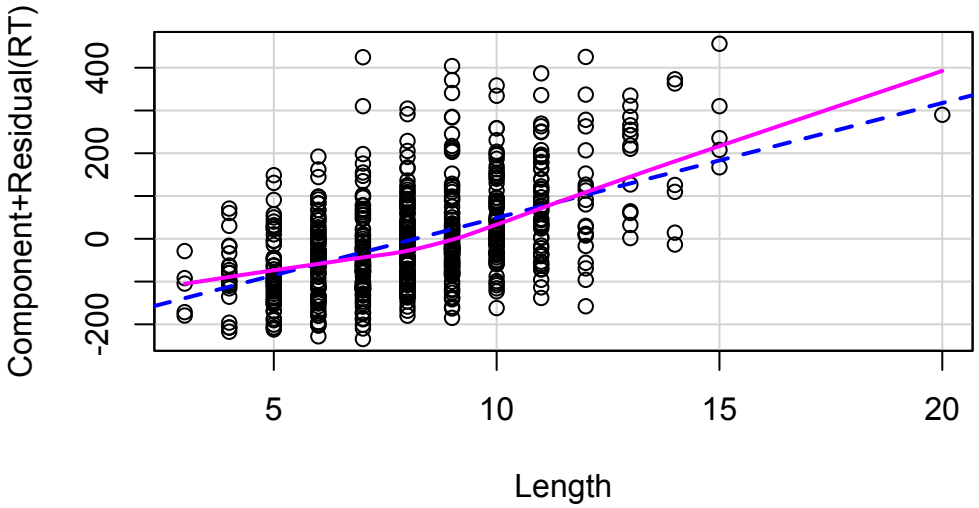

### Model assumptions and diagnostics

As a parametric method, linear regression makes numerous assumptions about the training data. It is, therefore, essential to run further tests to rule out possible violations. Among other things, the model assumptions include:

- A **linear relationship** between the response and the quantitative predictors: The residuals should not display a clear pattern. For this reason, it is recommended to use component residual plots (e.g., `crPlot()` from the `car` library) for the visual identification of potentially non-linear trends.

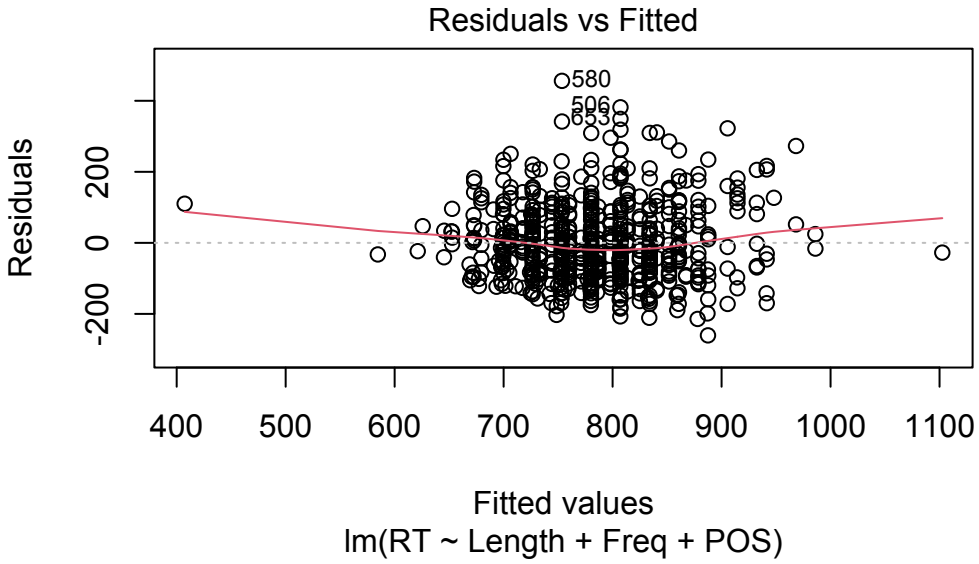
```
# pink line = main tendency vs. blue line = slope coefficients;
# some minor non-linearity can be observed

crPlot(rt.lm2, var = "Length") # potentially problematic
```




- **Homoscedasticity** (i.e., constant variance of the error terms; the opposite is called heteroscedasticity): Visually, a violation of this assumption becomes apparent if the residuals form a funnel-like shape. It is also possible to conduct a non-constant variance test `ncvTest()`: If it returns $p$-values $< 0.05$, it suggests non-constant variance.

```
plot(rt.lm2, which = 1)
```




```
ncvTest(rt.lm2) # significant, meaning that the error variance is not
 ↪  constant
```

```
Non-constant Variance Score Test
Variance formula: ~ fitted.values
Chisquare = 17.4935, Df = 1, p = 2.8829e-05
```

- **No multicollinearity**: Predictors should not be strongly correlated with each other. Strongly correlated predictors have inflated standard errors, which makes their individual coefficient estimates unstable and difficult to interpret (the predictive performance of the model as a whole is not necessarily affected). Another diagnostic measure are variance inflation factors (VIF-scores); predictors with VIF scores $> 5$ are potentially collinear. They can be computed using the `vif()` function.

```
vif(rt.lm2) # vif < 5 indicates that predictors are not collinear
```

```
           GVIF Df GVIF^(1/(2*Df))
Length 1.028568  1        1.014183
Freq   1.028803  1        1.014299
POS    1.021578  2        1.005351
```

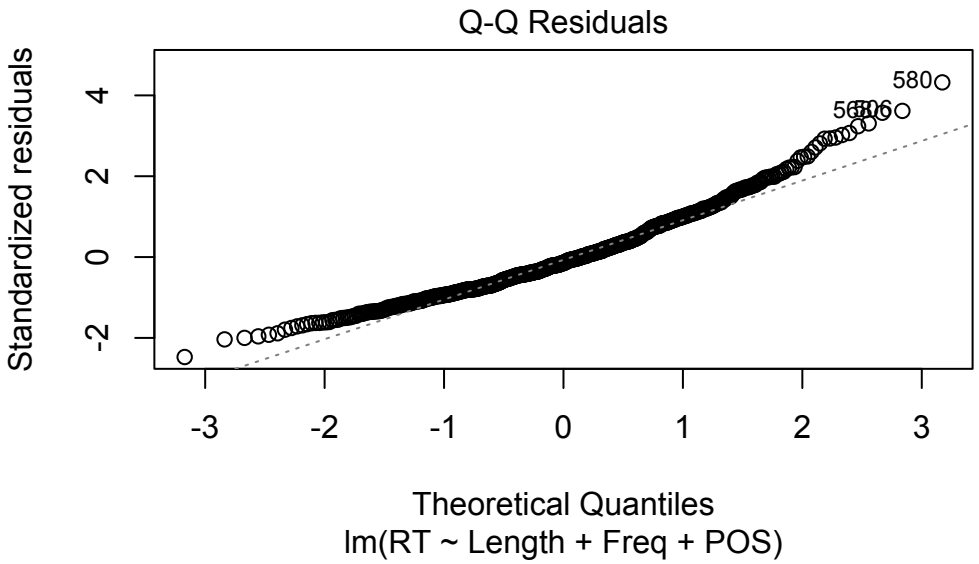
- **Normally distributed residuals**: The residuals should follow the normal distribution and be centered around 0:

$$\epsilon \sim N(0, \sigma^2) \tag{14}$$

Usually, a visual inspection using `qqnorm()` is sufficient, but the Shapiro-Wilk test `shapiro.test()` can also be run on the model residuals. Note that a $p$-value below 0.05 provides evidence for non-normality. For right-skewed response variables such as reaction times, a common remedy is to model the log-transformed response instead, e.g., `log(RT)` (cf. the histograms above).

- **Independent errors**: The observations (and hence their errors) should be independent of each other. If there are several observations per speaker or item, mixed-effects models are more appropriate (see 4.3 Mixed-effects regression).

```
plot(rt.lm2, which = 2)
```




```
shapiro.test(residuals(rt.lm2)) # residuals are not normally distributed
 ↪  because p < 0.05
```

```

    Shapiro-Wilk normality test

data:  residuals(rt.lm2)
W = 0.96713, p-value = 5.292e-11
```

## Model performance

### Test error

A crucial step in machine learning is evaluating how well our model performs, both on the data it was trained on and, more importantly, on completely new data. This helps us understand whether our model has learned meaningful patterns or simply memorised the training data; the latter phenomenon is known as **overfitting**. James et al. (2021: 24) explain that overfitting is a serious problem "because the fit obtained will not yield accurate estimates of the response on new observations that were not part of the original training data set", thus limiting both its theoretical and practical value.

We start by making predictions on both the training and the test data:

```
set.seed(123)

# Predict on training and test data
train_pred <- predict(rt.lm2, newdata = train_data)
test_pred <- predict(rt.lm2, newdata = test_data)
```

The preferred model is the one that minimises a loss metric such as the **Mean Squared Error** (MSE):

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2. \tag{15}$$

Here, $\hat{y}_i$ denotes the predicted response value for the $i$th observation. The closer the predictions are to the true response values, the lower the MSE. Since a model is fitted to its training data, the test MSE is usually somewhat higher than the training MSE. A **substantially** higher test MSE, however, is a typical sign of overfitting. Let's investigate:

```
# Training MSE
train_mse <- mean((train_data$RT - train_pred)^2)

print(train_mse)
```

```
[1] 11087.82
```

```
# Test MSE
test_mse <- mean((test_data$RT - test_pred)^2)

print(test_mse)
```

```
[1] 11704.49
```

The test MSE is lower than the training MSE, but only marginally so. There is little evidence of overfitting.

**i** Other regression loss metrics

A comprehensive overview of regression loss functions was compiled by Aryan Jadon, Avinash Patil, and Shruti Jadon (2022) in their arXiv preprint [Last accessed: June 29, 2025]. Some fairly common alternatives are the **Root Mean Squared Error** (RMSE)

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{16}$$

```
# Training RMSE
train_rmse <- sqrt(mean((train_data$RT - train_pred)^2))
print(train_rmse)
```

```
[1] 105.2987
```

```
# Test RMSE
test_rmse <- sqrt(mean((test_data$RT - test_pred)^2))
print(test_rmse)
```

```
[1] 108.1873
```

and the **Mean Absolute Error** (MAE)

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|. \tag{17}$$

```
# Training MAE
train_mae <- mean(abs(train_data$RT - train_pred))
print(train_mae)
```

```
[1] 82.56613
```

```
# Test MAE
test_mae <- mean(abs(test_data$RT - test_pred))
print(test_mae)
```

```
[1] 83.06574
```

### Cross-validation

Another common strategy for assessing a model's test accuracy is $k$**-fold cross-validation**. It is particularly useful if no separate test data are available. Essentially, it involves randomly dividing the data into $k$ equally sized subsets, known as *folds*. The model is then fitted $k$ times, each time on $k-1$ folds, and evaluated on the remaining fold. Each fold thus provides an estimate of the test error; their average is the cross-validated performance estimate:

$$CV_{(k)} = \frac{1}{k}\sum_{i=1}^{k} MSE_i \tag{18}$$

```
# Define training control
train.control <- trainControl(method = "cv", number = 10)

# Train the model
cv_model <- train(RT ~ Length + Freq + POS, data = ELP, method = "lm",
 ↪  trControl = train.control)

# Summarise the results (note that caret reports the RMSE rather than the
 ↪  MSE)
```

```
print(cv_model)
```

```
Linear Regression

880 samples
  3 predictor

No pre-processing
Resampling: Cross-Validated (10 fold)
Summary of sample sizes: 792, 792, 792, 792, 792, 792, ...
Resampling results:

  RMSE     Rsquared   MAE
  114.228  0.2805632  84.68323

Tuning parameter 'intercept' was held constant at a value of TRUE
```

```
# Print more detailed results
cv_model$results
```

```
  intercept    RMSE  Rsquared      MAE   RMSESD RsquaredSD   MAESD
1      TRUE 114.228 0.2805632 84.68323 29.21069 0.08671399 9.33676
```

```
# Check individual folds
cv_model$resample
```

```
       RMSE  Rsquared       MAE Resample
1  101.9075 0.3002226  78.44848   Fold01
2   90.7993 0.4098988  72.52993   Fold02
3  105.2724 0.3303321  81.68349   Fold03
4  104.8611 0.2439084  82.85119   Fold04
5  105.2912 0.2631391  82.64165   Fold05
6  111.5148 0.2584855  82.07014   Fold06
7  107.2410 0.4222355  79.07264   Fold07
8  105.4771 0.1927756  89.43880   Fold08
9  195.4565 0.1562914 106.39181   Fold09
10 114.4592 0.2283430  91.70414   Fold10
```

```
# See ?caret::train for an overview of available tuning parameters
```

## Exercises

Solutions

You can find the solutions to the exercises here.

### Tier 1

**Exercise 1.** Using the full ELP dataset, fit a simple linear regression model `m1` predicting reaction time (`RT`) from word frequency (`Freq`).

a. Fit the model using `lm()` and display the model summary.
b. Extract and interpret the coefficient for `Freq`.
c. Is the relationship between frequency and reaction time significant? How do you know?
d. Evaluate the explanatory power of the model.

**Exercise 2.** Using your frequency model from the previous exercise:

a. Manually calculate the predicted reaction time for a word with frequency = 5.
b. Use the `predict()` function to predict reaction times for the following word frequencies: 1, 10, 50, 500, 1000 (Tip: Store the frequencies as a column in a data frame).

### Tier 2

**Exercise 3.** We can use `anova()` to assess how the model fit changes when including or dropping predictors.

**Example**:

```
anova(rt.lm1, rt.lm2)
```

```
Analysis of Variance Table

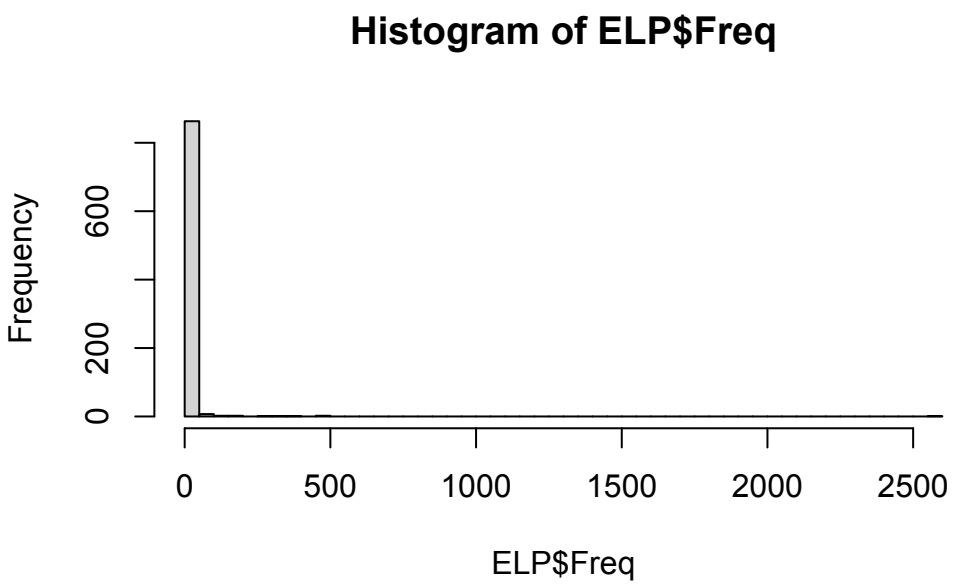
Model 1: RT ~ Length
Model 2: RT ~ Length + Freq + POS
  Res.Df     RSS Df Sum of Sq      F   Pr(>F)
1    658 7476218
2    655 7317963  3    158255 4.7216 0.002872 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The more complex model with `Length`, `Freq`, and `POS` is significantly better than the one only including `Length` ($F(3, 655) = 4.72, p < 0.01$).

Examine the impact of including `POS` in model `m1` from Exercise 1: Does it improve the fit?

**Exercise 4.** As is typical of word frequency distributions, the variable `Freq` is extremely skewed:

```
hist(ELP$Freq, breaks = 50)
```



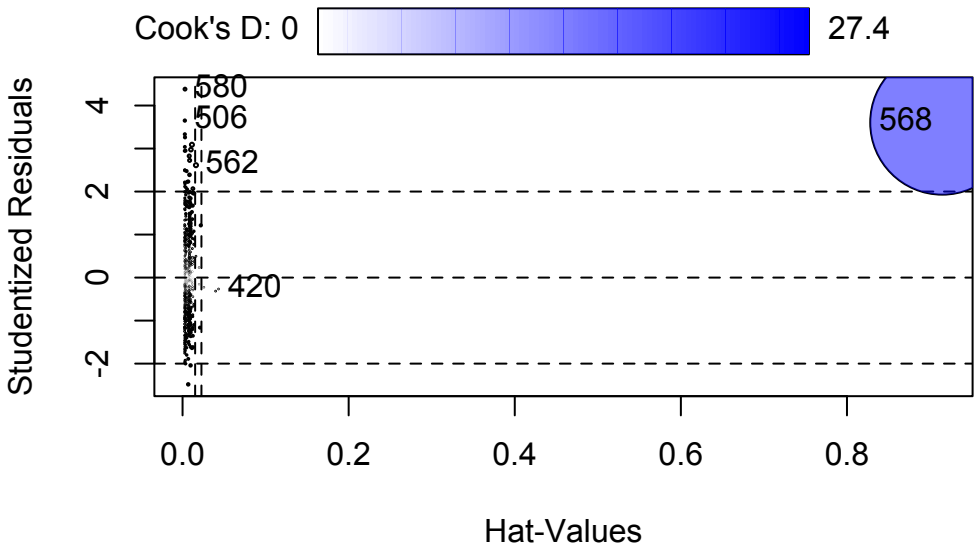

a. Transform the word frequencies using the natural logarithm (`log()`) and add the output to the ELP data frame as a separate column. Name this column `Log_Freq`.
b. Evaluate the following models in terms of their fit and performance. Which model would you select and why?

   - `RT ~ Freq + Length + POS`
   - `RT ~ Log_Freq + Length + POS`
   - `RT ~ Freq + Log_Freq + Length + POS`

### Tier 3

In the same way as outliers may 'drag' the sample mean into a certain direction, they may also skew the coefficient estimates in a regression model. The `influencePlot()` function from the `car` package makes it easy to identify overly influential observations (for details, see Levshina (2015: 153-155)).

```
influencePlot(rt.lm2) # labels the most noteworthy observations
```




```
       StudRes         Hat        CookD
420 -0.2653199 0.043321736 6.384495e-04
506  3.6499855 0.002739375 7.183908e-03
562  2.6133513 0.016126288 2.219077e-02
568  3.6030935 0.914760281 2.736361e+01
580  4.3813802 0.002942841 1.102549e-02
```

Potentially problematic observations are those with studentised residuals $> 2$ or $< -2$ and large hat values. Hat values measure an observation's leverage, i.e., how unusual its predictor values are and hence how strongly it can pull the regression line towards itself. Additionally, Cook's distances capture how much the coefficient estimations change when a specific observation is removed (cf. bubble size).

Inspect the observations and residuals corresponding to row numbers 568, 580, 506, 562, and 420 in the `train_data`, which was used to fit the `rt.lm2` model. What could be the cause of these unusually high residuals/Cook's distances?

**Follow-up:** Remove these five observations from the original ELP data.

- How do the coefficients change compared to `rt.lm2`? Does this change your substantive conclusions about the predictors?
- Perform $k$-fold cross-validation on the original ELP data and the ELP data without outliers using the `train()` function. To obtain a robust error estimate, use repeated 10-fold cross-validation with 30 repetitions (`trainControl(method = "repeatedcv", number = 10, repeats = 30)`). Does removing the outliers affect the model performance?

## References

Agresti, Alan, and Maria Kateri. 2022. *Foundations of Statistics for Data Scientists: With r and Python*. Boca Raton: CRC Press.

Balota, David A., Melvin J. Yap, Keith A. Hutchison, Michael J. Cortese, Brett Kessler, Bjorn Loftis, James H. Neely, Douglas L. Nelson, Greg B. Simpson, and Rebecca Treiman. 2007. "The English Lexicon Project." *Behavior Research Methods* 39 (3): 445–59. https://doi.org/10.3758/BF03193014.

Hastie, Trevor, Robert Tibshirani, and Jerome H. Friedman. 2017. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. New York, NY: Springer.

Heumann, Christian, Michael Schomaker, and Shalabh. 2022. *Introduction to Statistics and Data Analysis: With Exercises, Solutions and Applications in r*. 2nd ed. Cham: Springer. https://doi.org/10.1007/978-3-031-11833-3.

James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2021. *An Introduction to Statistical Learning: With Applications in r*. New York: Springer. https://doi.org/10.1007/978-1-0716-1418-1.

Levshina, Natalia. 2015. *How to Do Linguistics with r: Data Exploration and Statistical Analysis*. Amsterdam; Philadelphia: John Benjamins Publishing Company.

Winter, Bodo. 2020. *Statistics for Linguists: An Introduction Using r*. New York; London: Routledge.